Manual de Despliegue

Proyecto Django en AlwaysData

Conexión con GitHub · Variables de Entorno · MySQL · Archivos Estáticos

REQUISITOS PREVIOS

- Proyecto Django terminado.
- Proyecto alojado en GitHub.

INSTRUCTOR: FELIPE CUEVAS

¿Qué vamos a lograr en este manual?

Hasta ahora tu proyecto Django corre perfectamente en tu computador local. ¡Genial! Pero ese servidor sólo vive en tu máquina. El mundo exterior no puede verlo. El objetivo de este manual es llevar tu proyecto a Internet, donde cualquier persona con el link podrá acceder a él.

Para eso usaremos AlwaysData, una plataforma de hosting que tiene una capa gratuita muy conveniente para proyectos académicos. Lo haremos paso a paso, explicando qué hace cada configuración para que no sea sólo un copy-paste, sino que entiendas el razonamiento detrás de cada decisión.

Conceptos clave antes de empezar

AlwaysData

Es un proveedor de hosting que permite ejecutar aplicaciones web Python/Django en sus servidores.

Tu código en GitHub

Tu código vive en GitHub, y AlwaysData lo descarga desde ahí cada vez que haces un despliegue.

Control de versiones

Esto se llama despliegue desde control de versiones y es el flujo estándar de la industria.



PARTE 1 · Crear cuenta en AlwaysData

1) Registro en la plataforma

1. Abre tu navegador y visita: **https://www.alwaysdata.com**
2. Haz clic en el botón «Register» (Registrarse).
3. Completa el formulario con tu correo, nombre de usuario y contraseña.
4. Verifica tu correo electrónico haciendo clic en el enlace que recibirás.
5. Al ingresar por primera vez, la plataforma te pedirá crear tu primer «account» (cuenta de hosting). Elige la opción gratuita («free plan»).

¿Qué es un 'account' en AlwaysData? AlwaysData distingue entre tu usuario (login) y tus cuentas de hosting. Una cuenta de hosting es el «contenedor» donde vivirá tu sitio web, tu base de datos y tus archivos. Con el plan gratuito puedes tener una cuenta con 100 MB de almacenamiento, suficiente para un proyecto académico.

2) Familiarizarse con el panel de administración

Una vez dentro, verás el panel de administración. Las secciones que más usarás son:



Web > Sites

Crear y configurar el sitio Django (la app principal).



Databases > MySQL

Crear la base de datos MySQL de producción.



Environment > Variables

Configurar las variables de entorno secretas.



SSH

Conectarse al servidor para comandos manuales.



Files

Explorar el sistema de archivos del servidor.

PARTE 2 · Preparar el repositorio GitHub

Antes de configurar AlwaysData, tu repositorio debe tener ciertos archivos que el servidor necesitará para instalar dependencias y entender cómo ejecutar el proyecto.

3) Crear el archivo requirements.txt

- Este archivo le dice al servidor qué paquetes de Python debe instalar. Sin él, tu proyecto no funcionará en producción.
- Ejecuta este comando en la raíz de tu proyecto local (con el entorno virtual activado):

```
pip freeze > requirements.txt
```

El resultado será un archivo como este (los números de versión pueden variar):

```
Django==5.0.4
mysqlclient==2.2.4
python-decouple==3.8
Pillow==10.3.0
gunicorn==22.0.0
```

Importante: incluye gunicorn y mysqlclient
gunicorn es el servidor WSGI que AlwaysData usa para ejecutar Django en producción (en local usamos `runserver`, pero ese comando no está pensado para producción).
mysqlclient es el conector entre Django y MySQL. Asegúrate de que ambos aparezcan en tu `requirements.txt`.

PARTE 2 · Preparar el repositorio GitHub

4) Crear el archivo .env para desarrollo local (no lo subas a GitHub)

La idea es no hardcodear secretos directamente en `settings.py`. Usaremos la librería `python-decouple` para leer variables desde un archivo `.env` en local y desde variables de entorno del sistema en producción.

Instala python-decouple si no lo tienes:

```
pip install python-decouple
```

Crea el archivo .env en la raíz del proyecto:

```
# .env ← Este archivo NUNCA debe subirse a GitHub
SECRET_KEY=tu-clave-secreta-de-django-aqui
DEBUG=True
DB_NAME=nombre_de_tu_bd_local
DB_USER=root
DB_PASSWORD=tu_password_local
DB_HOST=localhost
DB_PORT=3306
```

Agrega .env a tu archivo .gitignore para que nunca se suba al repositorio:

```
# .gitignore
.env
__pycache__/
*.pyc
db.sqlite3
.venv/
```

El archivo `.env` contiene tus secretos locales. El archivo `.gitignore` garantiza que Git lo ignore y nunca lo suba a GitHub.

PARTE 2 · Preparar el repositorio GitHub

5) Modificar settings.py para usar python-decouple

Reemplaza las configuraciones sensibles en tu `settings.py` con llamadas a `config()`:

```
# GestionEscolar/settings.py
from decouple import config

# La clave secreta ya no está escrita aquí directamente
SECRET_KEY = config('SECRET_KEY')

# En producción será False, en local True
DEBUG = config('DEBUG', default=False, cast=bool)
ALLOWED_HOSTS = ['tu-cuenta.alwaysdata.net', 'localhost', '127.0.0.1']

# Configuración de base de datos dinámica
DATABASES = {
 'default': {
 'ENGINE': 'django.db.backends.mysql',
 'NAME': config('DB_NAME'),
 'USER': config('DB_USER'),
 'PASSWORD': config('DB_PASSWORD'),
 'HOST': config('DB_HOST'),
 'PORT': config('DB_PORT', default='3306'),
 }
}
```

PARTE 2 · Preparar el repositorio GitHub

6) Configurar archivos estáticos en settings.py

En producción, Django no sirve los archivos estáticos directamente (eso lo hace el servidor web). Necesitas indicarle dónde colectarlos:

```
import os

# Carpeta donde collectstatic copiará TODO lo estático
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')

# URL pública desde donde se accede a los estáticos
STATIC_URL = '/static/'

# Carpetas adicionales donde buscará estáticos en desarrollo
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'static'),
]

# Directorio global de templates
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')],  # <-- carpeta global
        'APP_DIRS': True,
        ...
    }
]
```

¿Por qué STATIC_ROOT y STATICFILES_DIRS son distintos?

STATICFILES_DIRS

Le dice a Django dónde buscar tus archivos estáticos **durante el desarrollo**.

STATIC_ROOT

Es el destino donde el comando `collectstatic` copiará **todos** los estáticos (incluyendo los del admin de Django) en un solo lugar para que el servidor web los sirva eficientemente. En producción, el servidor web (como Apache o Nginx) apunta directamente a `STATIC_ROOT`.

PARTE 3 · Configurar MySQL en AlwaysData

En producción no usaremos SQLite porque no está pensado para entornos con múltiples usuarios concurrentes. Usaremos **MySQL**, que AlwaysData ofrece de forma nativa.

7) Crear la base de datos MySQL

1. En el panel de AlwaysData, ve a la sección «Databases» y luego a «MySQL».
2. Haz clic en «Add a database».
3. Asigna un nombre descriptivo, por ejemplo: `gestion_escolar_db`.
4. Haz clic en «Submit». AlwaysData creará la base de datos automáticamente.

El nombre real de tu base de datos tiene un prefijo

AlwaysData prefija el nombre de tu base de datos con el nombre de tu cuenta. Si tu cuenta se llama `miescuela` y la BD se llama `gestion_escolar_db`, el nombre real será `miescuela_gestion_escolar_db`. Anota este nombre completo, lo necesitarás como valor de `DB_NAME`.

PARTE 3 · Configurar MySQL en AlwaysData

8) Obtener las credenciales de conexión

Una vez creada la base de datos, necesitas los datos de conexión. En AlwaysData con el plan gratuito:

Variable	Valor	Dónde encontrarlo
`DB_NAME`	`cuenta_nombre_bd`	Databases > MySQL (columna Name)
`DB_USER`	`cuenta` (tu nombre de cuenta)	Settings > General > Account name
`DB_PASSWORD`	tu password de la cuenta	El que definiste al registrarte
`DB_HOST`	`mysql-cuenta.alwaysdata.net`	Databases > MySQL (columna Host)
`DB_PORT`	`3306`	Puerto estándar de MySQL

PARTE 4 · Configurar Variables de Entorno

Las variables de entorno son la manera segura de manejar secretos en producción. En lugar de escribir contraseñas en el código (¡nunca hagas eso!), el servidor operativo las inyecta al proceso de Python en tiempo de ejecución.

9) Agregar variables de entorno en AlwaysData

1. En el panel de AlwaysData, navega a «Environment» y luego «Variables».
2. Haz clic en «Add a variable» para agregar cada una de las siguientes:

```
SECRET_KEY  = tu-clave-secreta-de-50-caracteres-generada-con-django
DEBUG       = False
DB_NAME     = cuenta_gestion_escolar_db
DB_USER     = nombre_de_tu_cuenta_alwaysdata
DB_PASSWORD = tu_password_de_alwaysdata
DB_HOST     = mysql-cuenta.alwaysdata.net
DB_PORT     = 3306
```

¿Cómo generar una SECRET_KEY segura?

Nunca reutilices la `SECRET_KEY` de desarrollo. Para generar una nueva, ejecuta este comando en cualquier terminal con Python:

```
python -c "from
django.core.management.utils
import get_random_secret_key;
print(get_random_secret_key())"
```

¿Por qué DEBUG=False en producción?

Con `DEBUG=True` Django muestra pantallas de error detalladas que revelan rutas de archivos, variables de entorno y fragmentos de código. Un atacante podría usar esa información. En producción **SIEMPRE** debe ser `False`. Con `DEBUG=False`, los errores muestran sólo una página genérica al usuario.

PARTE 5 · Conectar GitHub y desplegar el sitio

---

10) Crear el sitio web en AlwaysData

1. Ve a «Web» y luego «Sites» en el panel principal.
2. Haz clic en «Add a site».
3. Configura los campos de la siguiente manera:

1	**Addresses** `tu-cuenta.alwaysdata.net` (o tu dominio propio).
2	**Type** `Python WSGI`
3	**Application path** El directorio raíz del proyecto (ver paso 11).
4	**WSGI module** `GestionEscolar.wsgi`
5	**Python version** `3.11` o superior.

11) Conectar el repositorio GitHub via SSH

AlwaysData te da acceso SSH a tu servidor. Usaremos esa conexión para clonar tu repositorio. Primero necesitas generar una clave SSH en el servidor y agregarla a GitHub.

11.1 - Conectarse por SSH al servidor

Desde tu terminal local:

```
ssh tu-cuenta@ssh-tu-cuenta.alwaysdata.net
```

Una vez dentro del servidor, verás un prompt como: `tu-cuenta@ssh ~$`

11.2 - Generar clave SSH en el servidor

```
# Generar par de claves RSA
ssh-keygen -t ed25519 -C 'alwaysdata-deploy'

# Presiona Enter en todas las preguntas (sin passphrase para despliegues)

# Ver la clave pública que deberás copiar
cat ~/.ssh/id_ed25519.pub
```

Copia toda la salida del último comando. Se verá así:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... alwaysdata-deploy
```

11.3 - Agregar la clave a GitHub

1. Ve a tu repositorio en GitHub.
2. Entra a «Settings» > «Deploy keys» > «Add deploy key».
3. Pega la clave pública que copiaste y marca la opción «Allow write access» si necesitas hacer push desde el servidor (generalmente no es necesario).
4. Haz clic en «Add key».

11.4 - Clonar el repositorio en el servidor

De regreso en la terminal SSH del servidor, navega al directorio principal y clona el proyecto:

```
# Ir al directorio home
cd ~

# Clonar usando SSH (no HTTPS, para evitar pedir contraseña)
git clone git@github.com:tu-usuario/tu-repositorio.git

# Ingresar al directorio del proyecto
cd tu-repositorio

# Verificar que estás en la rama correcta
git branch
```

12) Instalar dependencias en el servidor

En el servidor, crea un entorno virtual e instala los paquetes del proyecto:

```
# Crear entorno virtual dentro del proyecto
python3 -m venv .venv

# Activar el entorno virtual
source .venv/bin/activate

# Instalar todas las dependencias
pip install -r requirements.txt
```

Si mysqlclient falla al instalar

En algunos servidores, `mysqlclient` requiere librerías del sistema. Si falla, prueba con:

`pip install PyMySQL`

Y luego agrega en tu `__init__.py` del proyecto:

```
import pymysql
pymysql.install_as_MySQLdb()
```

Esta es una alternativa 100% Python que no requiere librerías del sistema.

IMPORTANTE — ¿Y después del primer despliegue, qué hago?

Todo lo que configuraste en esta sección es un **proceso único** que solo realizarás una vez: generar la clave SSH, agregarla a GitHub y clonar el repositorio en el servidor. No necesitas repetirlo nunca más.

A partir de ahora, cada vez que quieras publicar nuevos cambios en producción, el flujo de trabajo diario es mucho más simple: haces tu **git push** desde local como siempre y luego ejecutas un **git pull** desde el servidor. Este proceso repetible está explicado paso a paso al final del manual en la **PARTE 9 · Flujo de trabajo para actualizaciones**.

PARTE 6 · Configurar archivos estáticos y templates

13) Configurar la ruta de archivos estáticos en AlwaysData

Recuerdas el `STATIC_ROOT` que configuramos en `settings.py`. Ahora necesitas indicarle a AlwaysData que esa carpeta existe y debe servirse directamente:

1. En el panel, ve a «Web» > «Sites» y edita tu sitio.
2. Busca la sección «Static paths» o «Statics».
3. Agrega una entrada con los siguientes valores:

URL path

`/static/`

File path

`/home/tu-cuenta/tu-repositorio/staticfiles` *(staticfiles es el directorio generado por collectstatic)*

Ahora ejecuta el comando `collectstatic` en el servidor para poblar esa carpeta:

```
# Asegúrate de tener el entorno virtual activado
source .venv/bin/activate

# Recolectar todos los archivos estáticos
python manage.py collectstatic --noinput

# Deberías ver algo como:
# 127 static files copied to '/home/tu-cuenta/tu-repo/staticfiles'
```

¿Qué hace exactamente collectstatic?
Este comando recorre todas las apps instaladas (incluida la app admin de Django) y copia todos sus archivos CSS, JS e imágenes a la carpeta `STATIC_ROOT`. Así el servidor web tiene un único directorio desde donde servir todo lo estático, sin necesidad de que Django intervenga en cada petición de imagen o CSS.

14) Verificar la carpeta global de templates

Tu proyecto ya tiene una carpeta `templates` en la raíz con los templates globales (`base.html`, `login.html`, etc.). Verifica que Django la encuentra correctamente. En el servidor, ejecuta:

```
python manage.py shell

# Dentro del shell de Django:
from django.template.loaders.app_directories import Loader
from django.conf import settings
print(settings.TEMPLATES[0]['DIRS'])

# Deberías ver la ruta absoluta a tu carpeta templates
# Ejemplo: ['/home/tu-cuenta/tu-repo/templates']
exit()
```



Este proceso confirma que Django puede localizar correctamente los templates globales del proyecto antes de continuar con las migraciones.

PARTE 7 · Ejecutar migraciones y crear superusuario

15) Aplicar las migraciones a MySQL

Las migraciones crean las tablas en tu base de datos MySQL de producción. Es el equivalente a tener el esquema de la base de datos actualizado. Ejecuta en el servidor:

```
# Primero verifica que la conexión a MySQL funciona
python manage.py dbshell

# Si se abre el prompt de MySQL, ¡la conexión funciona! Escribe \q para salir

# Aplicar todas las migraciones
python manage.py migrate

# Deberías ver una lista de operaciones como:
# Applying contenttypes.0001_initial... OK
# Applying auth.0001_initial... OK
# Applying admin.0001_initial... OK
# etc.
```

Error 'Access denied for user'
Si `migrate` falla con un error de acceso, revisa que las variables de entorno estén bien configuradas. Puedes verificarlas ejecutando `echo $DB_PASSWORD` en la terminal SSH. Si aparece vacío, las variables no se están cargando correctamente. Vuelve al **Paso 9** y confirma que las guardaste en el panel de AlwaysData.

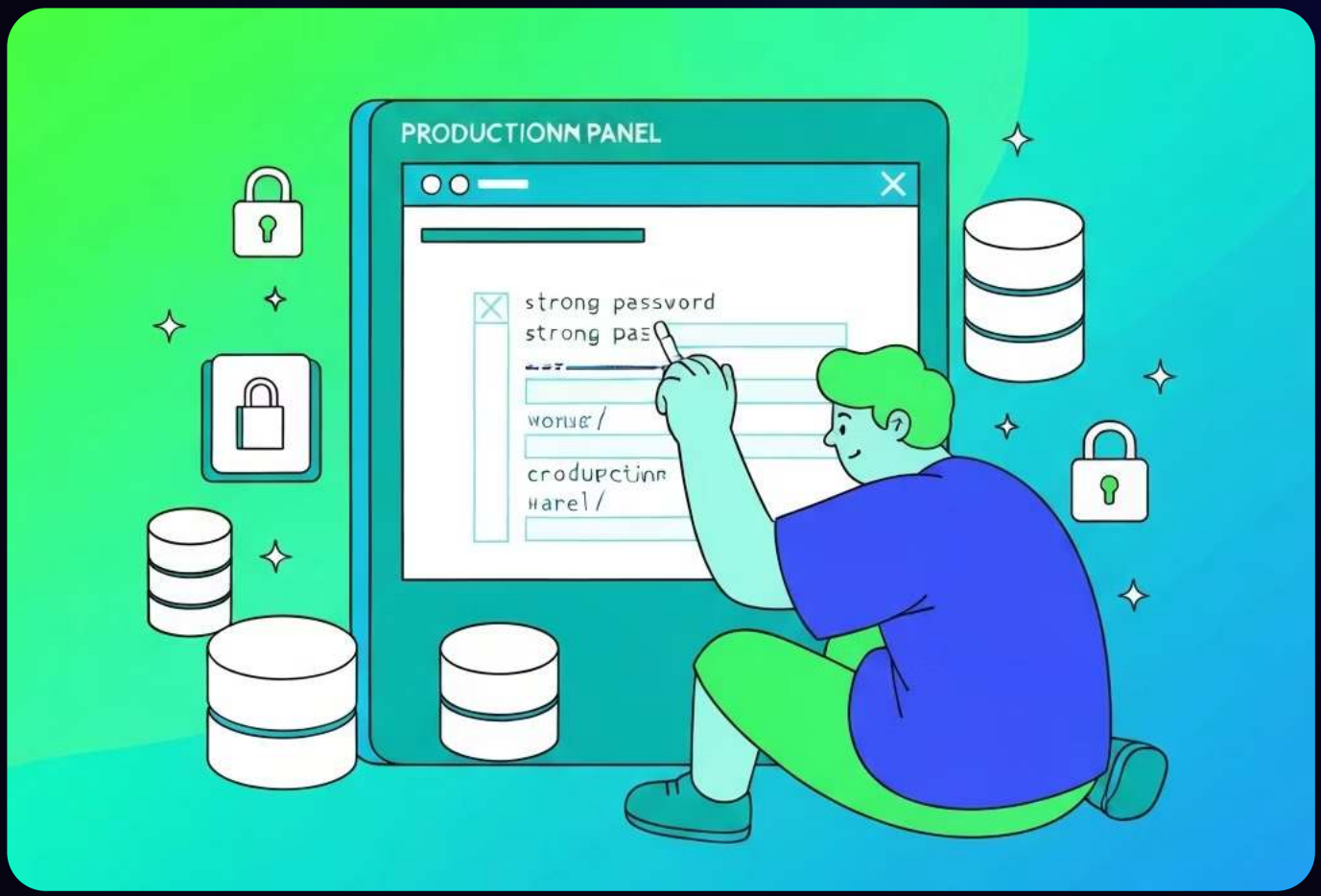
16) Crear el superusuario de producción

Necesitas un usuario administrador para acceder al panel `/admin/` en producción:

```
python manage.py createsuperuser

# El sistema te pedirá:
# Username: admin_produccion
# Email: tu@email.com
# Password: (elige una contraseña segura)
# Password (again): (repite la contraseña)
```

Usa credenciales distintas a las de desarrollo
Nunca uses `admin / 1234` en producción. Elige un nombre de usuario que no sea obvio y una contraseña larga con mayúsculas, números y símbolos. Las bases de datos en producción son un objetivo frecuente de ataques automatizados.



PARTE 8 · Configuración final del sitio WSGI

---

17) Verificar la configuración del módulo WSGI

El archivo `wsgi.py` es el punto de entrada que AlwaysData usa para comunicarse con Django. Verifica que apunta correctamente al módulo de settings:

```
# GestionEscolar/wsgi.py (ya existe en tu proyecto)
import os
from django.core.wsgi import get_wsgi_application

# Este valor debe coincidir con el módulo de settings de tu proyecto
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'GestionEscolar.settings')

application = get_wsgi_application()
```

18) Reiniciar el sitio en AlwaysData

Cada vez que hagas cambios en el código o la configuración, debes reiniciar el proceso para que AlwaysData aplique los cambios:

1. Ve a «Web» > «Sites» en el panel de AlwaysData.
2. Haz clic en el botón de reinicio (ícono de recarga) junto a tu sitio.
3. Espera unos segundos y luego visita tu URL: **https://tu-cuenta.alwaysdata.net**

> **¿El sitio muestra error 500?**
> Un error 500 en producción (con `DEBUG=False`) no muestra detalles. Para diagnosticar, revisa los logs de AlwaysData en «Logs» > «Site logs». También puedes activar temporalmente `DEBUG=True` en las variables de entorno, ver el error y luego volver a `False`.

PARTE 9 · Flujo de trabajo para actualizaciones

Una vez que el proyecto está funcionando, el flujo para publicar cambios es simple y repetible:

1. **Editar y probar en local**

   Editas el código en tu computador local y lo pruebas con `runserver`.

2. **Commit y push a GitHub**

   `git add . && git commit -m 'descripción' && git push`

3. **Conectarse al servidor por SSH**

   `ssh tu-cuenta@ssh-tu-cuenta.alwaysdata.net`

4. **Hacer pull en el servidor**

   `cd tu-repo && git pull`

5. **Migraciones y estáticos (si aplica)**

   Si hay nuevas migraciones: `python manage.py migrate`
   Si cambiaron archivos estáticos: `python manage.py collectstatic --noinput`

6. **Reiniciar el sitio**

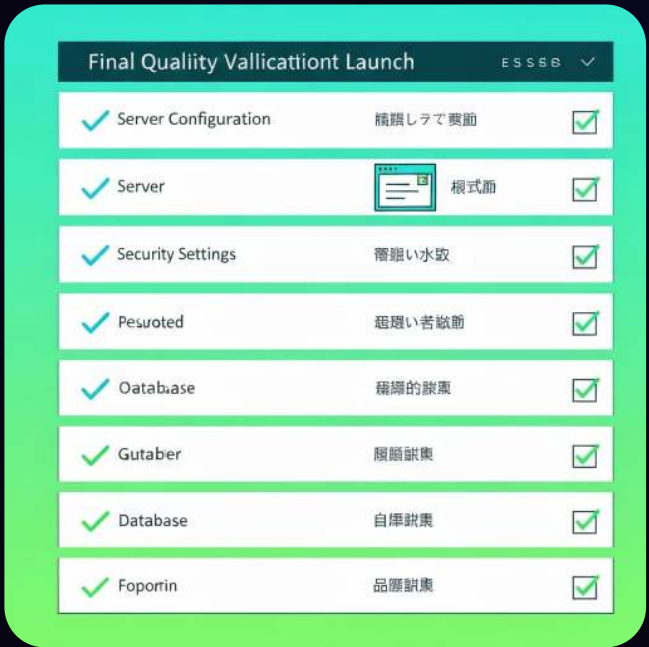
   Reinicia el sitio desde el panel de AlwaysData (o con `touch wsgi.py`).

PARTE 10 · Checklist de verificación final

Antes de declarar el despliegue como exitoso, revisa que todos estos puntos estén cumplidos:

- El sitio carga en la URL de AlwaysData sin errores.
- La página de login y las rutas públicas funcionan.
- El admin de Django (`/admin/`) es accesible con el superusuario creado.
- Los archivos CSS y JS cargan correctamente (sin el ícono roto del navegador).
- El formulario de contacto u otro formulario funciona y guarda en MySQL.





- El archivo `.env` NO está visible en el repositorio público de GitHub.
- `DEBUG=False` está configurado en las variables de entorno de producción.
- La `SECRET_KEY` de producción es diferente a la de desarrollo.
- Las migraciones se aplicaron sin errores (`python manage.py showmigrations`).
- Los templates globales (`base.html`, `login.html`) renderizan correctamente.

PARTE 11 · Errores comunes y cómo resolverlos

❌ Error: ModuleNotFoundError — No module named 'X'

Causa y solución

El entorno virtual no está activado o falta instalar el paquete.

Solución: asegúrate de haber ejecutado `source .venv/bin/activate` antes de `pip install -r requirements.txt` y verifica que el módulo WSGI de AlwaysData apunta a tu entorno virtual.

❌ Error: django.db.OperationalError — Access denied for user

Causa y solución

Las credenciales de MySQL son incorrectas o las variables de entorno no se están cargando.

Verifica en el panel de AlwaysData > Environment > Variables que los valores son exactamente correctos, sin espacios extra. Recuerda que el nombre de la BD tiene el prefijo de tu cuenta.

❌ Error: Static files no cargan (CSS roto)

Causa y solución

Olvidaste ejecutar `python manage.py collectstatic` o la ruta de Static paths en el sitio no coincide con `STATIC_ROOT`.

Verifica que `STATIC_URL='/static/'` en `settings.py` coincida con el URL path configurado en AlwaysData, y que `STATIC_ROOT` apunte a la carpeta `staticfiles` dentro de tu proyecto.

❌ Error: TemplateDoesNotExist

Causa y solución

Django no encuentra el template.

Verifica que `TEMPLATES['DIRS']` en `settings.py` incluye la ruta absoluta a tu carpeta `templates` global usando `os.path.join(BASE_DIR, 'templates')`. Asegúrate también de que los templates de las apps estén dentro de `app/templates/app/nombre.html`.

Conclusión

¡Tu proyecto Django está en producción!

> *Ahora cualquier persona en el mundo puede acceder a tu aplicación.*

Has completado el proceso completo de despliegue. Cada paso que realizaste sigue los estándares de la industria: separación de secretos con variables de entorno, gestión de dependencias con `requirements.txt`, base de datos MySQL en lugar de SQLite, y servicio de archivos estáticos desacoplado de Django.

INSTRUCTOR: FELIPE CUEVAS